How-To Library

Informatica

Intelligent Streaming Performance Tuning Guidelines

© Copyright Informatica LLC 2017, 2022. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Abstract

You can tune Intelligent Streaming for better performance. This article provides recommendations that you can use to tune hardware, memory, Spark configuration, and mapping configuration.

Supported Versions

- Informatica Intelligent Streaming 10.1.1

Table of Contents

Overview. 2
Plan Your Environment and Hardware. 2
Tune Memory. 3
Tune Spark Parameters. 3
Tune the Mapping. 5
Recommendations for Designing a Mapping. 5
Recommendations For Transformations. 5
Tune the External Data Ingestion System. 6
General Recommendations. 6
Monitor Your Production Environment. 7

Overview

Use Informatica Intelligent Streaming mappings to collect streaming data, build the business logic for the data, and push the logic to a Spark engine for processing. The Spark engine uses Spark Streaming to process data. The Spark engine reads the data, divides the data into micro batches, and publishes it.

To optimize the performance of Intelligent Streaming and your system, perform the following tasks:

- Plan your environment and hardware.
- Tune memory.
- Tune Spark parameters.
- Tune the mapping.
- Tune the external data ingestion system.
- Monitor your environment.

Plan Your Environment and Hardware

To optimize the performance, acquire the right type of hardware and design the right architecture for your Intelligent Streaming environment.

Perform the following tasks:

- Procure hardware based on sizing recommendations.
- Procure the type of hardware that you need such as virtual or physical hardware, whether on the cloud or on-premises, based on business needs. Tune the hardware such that it performs to its full capacity and does not lead to loss of productivity and business opportunities.

- Determine your streaming data needs, such as the number of mappings required and maximum number messages to process per second.
- Get a high speed network interface card to optimize performance. Apache recommends that you use an Ethernet card that supports 10 GB or more for faster performance.
- Incorporate more disks for each data node. Apache recommends that you use between 4 and 8 disks without RAID for every data node. To avoid unnecessary writes in a Linux system, mount the system with the `noatime` option.
- After you procure the hardware, perform the following tasks:
  - Ensure that the CPU is operating at the highest frequency.
  - Disable variable frequency for the CPU.
  - Set the ring size buffer of the network interface card to the preset maximum.
  - Disable huge page compaction.
  - Set the swappiness value to 0.

Tune Memory

Tune memory to avoid memory overheads and optimize performance. Some of the reasons for memory overhead include network buffers, off heap processing, and other DirectByteBuffer allocations.

To optimize performance, perform the following tasks:

- Consider increasing the number of executors to perform memory intensive tasks. For example, to perform a lookup on data greater than 500 MB, increase the number of executors.
- Configure a heap size such that the time spent on garbage collection is less than 5% of total time spent. Configuring a high heap size might result in long or delayed garbage collection pauses. Configuring a low heap size might result in frequent garbage collection pauses.

  To configure the heap size on Spark, configure the `spark.driver.memory` and `spark.executor.memory` parameters.
- Use the default parallel collector for garbage collection if the heap size is 16 GB to 32 GB and G1GC collector if the heap size exceeds 32 GB.
- In addition to heap memory, an executor also has an additional non-heap memory known as `MemoryOverHead`. The default value of `MemoryOverHead` is 384. If the executor RSS memory exceeds this value, YARN kills the process. To increase the `MemoryOverHead` value configure the `spark.yarn.driver.memoryOverhead` and `spark.yarn.executor.memoryOverhead` parameters in the `hadoop.Env.properties` file. Specify values of 4 GB for both the parameters.

Tune Spark Parameters

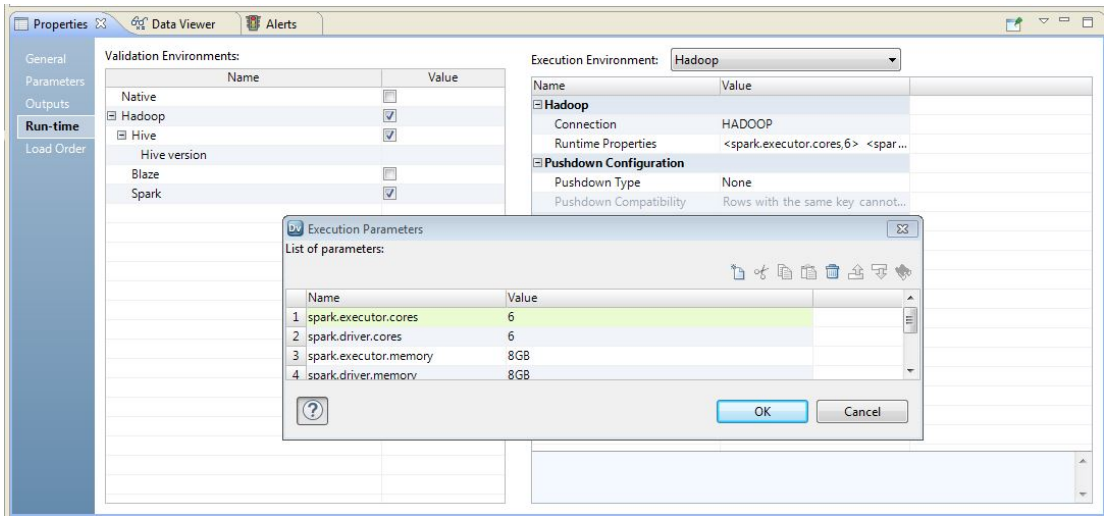
To optimize Intelligent Streaming performance, tune Spark parameters in the `hadoopEnv.properties` file. To tune Spark parameters for specific mappings, configure the execution parameters of the Streaming mapping Run-time

properties in the Developer tool. If you tune the parameters in the `hadoopEnv.properties` file, the configuration applies to all mappings that you create.

You can configure the following parameters based on the input data rate and mapping complexity and concurrency of mappings:

Parameter	Description
`spark.executor.cores`	The number of cores to use on each executor. Specify 3 to 4 cores for each executor. Specifying a higher number of cores might lead to performance degradation.
`spark.driver.cores`	The number of cores to use for the driver process. Specify 6 cores.
`spark.executor.memory`	The amount of memory to use for each executor process. Specify a value of 8 GB.
`spark.driver.memory`	The amount of memory to use for the driver process. Specify a value of 8 GB.
`spark.executor.instances`	The total number of executors to be started. This number depends on number of machines in the cluster, memory allocated, and cores per machine.
`spark.sql.shuffle.partitions`	The total number of partitions used for a SQL shuffle operation. Specify a value that equals the total number of executor cores.
`spark.rdd.compress`	Specifies whether to compress the serialized partitions in the RDD. This can save substantial space, but at the cost of CPU time. Optionally, to compress the serialized partitions, set the `spark.rdd.compress` execution parameter to true.
`spark.streaming.backpresure.enabled`	Allows Spark Streaming to control the receiving rate so that the system receives only as fast as the system can process. Set this parameter to true for JMS receivers.

The following image shows the execution parameters that you can configure as part of the Streaming mapping Run-time properties:



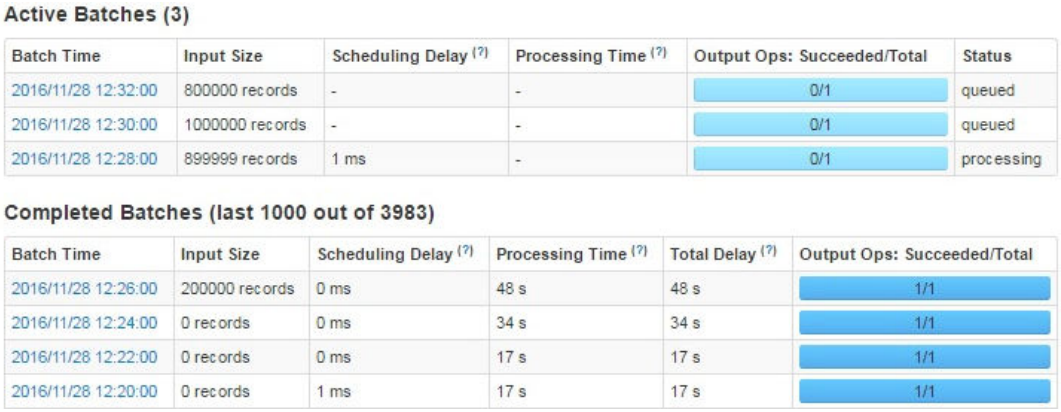
Tune the Mapping

To tune the mapping, use general guidelines or the recommendations based on the transformations that you use in the mappings.

Recommendations for Designing a Mapping

Consider the following recommendations while designing a mapping:

- Define a batch interval of moderate size to avoid message queueing as queueing can negatively impact performance. For every batch, Spark creates an RDD. If you configure a small interval of two seconds, Spark creates a batch every two seconds, and if you configure an interval of one hour, Spark accumulates data for one hour.
  The following image shows an example of job queue up based on batch interval:



Active Batches (3)

Batch Time	Input Size	Scheduling Delay (?)	Processing Time (?)	Output Ops: Succeeded/Total	Status
2016/11/28 12:32:00	800000 records	-	-	0/1	queued
2016/11/28 12:30:00	1000000 records	-	-	0/1	queued
2016/11/28 12:28:00	899999 records	1 ms	-	0/1	processing

Completed Batches (last 1000 out of 3983)

Batch Time	Input Size	Scheduling Delay (?)	Processing Time (?)	Total Delay (?)	Output Ops: Succeeded/Total
2016/11/28 12:26:00	200000 records	0 ms	48 s	48 s	1/1
2016/11/28 12:24:00	0 records	0 ms	34 s	34 s	1/1
2016/11/28 12:22:00	0 records	0 ms	17 s	17 s	1/1
2016/11/28 12:20:00	0 records	1 ms	17 s	17 s	1/1

- Define an appropriate precision for all types of fields. For example, if you define a string field of length five characters, additional characters are truncated which results in additional processing. Define a precision for each field keeping in mind the maximum length it can go up to.
- Avoid data type conversions, such as conversions from integer to string.

Recommendations For Transformations

Consider the following recommendations for tuning mapping performance based on the type of transformations that you use in the mapping:

Filter transformations

To discard messages that are not needed, use a Filter transformation as the first transformation in a mapping.

Lookup transformations

You can improve the performance of Lookup transformations in Streaming mappings by configuring the following properties:

- Advanced properties of the Lookup transformation. Enable the **Ignore null values that match** option.
- Execution parameters in the Streaming mapping Run-time properties. Configure the following properties: `infaspark.lookup.persist.enabled`, `infaspark.lookup.repartition.partitions`, `spark.rdd.compress`, and `spark.streaming.unpersist` properties.
- Custom properties of the Data Integration Service. Configure the `ExecutionContextOptions.Spark.PersistLevel` property.

For more information about tuning the performance of a Lookup transformation, see the Informatica How-to-Library article Performance Tuning Guidelines for Lookup Transformations in a Streaming Mappings.

Window transformations

You can improve the performance of Window transformations in Streaming mappings by configuring the following properties:

- Batch interval. Specify a batch interval that is near the slide interval or tumbling interval.
- Slide interval. Reduce the ratio of window to slide interval.

For example, if you configure a batch interval of 20 seconds and a slide interval of one second, the ratio is 20 seconds. This means that for every batch, the RDD computation happens 20 times.

Instead, if you configure a batch Interval of 20 seconds and slide interval of five seconds, the ratio is four seconds, the RDD computation happens four times. This reduces the complexity of the computation.

Joiner transformations

When you use Joiner transformations in Streaming mappings, verify that you are reading unique data.

Tune the External Data Ingestion System

Consider the following recommendations for tuning Kafka producers:

- Configure the Kafka cluster such that Intelligent Streaming can produce and consume messages at the needed message ingestion rate.
- To increase the rate of message consumption in Intelligent Streaming, increase the number of Kafka brokers in the Kafka cluster and in the Kafka connection.
- Increase the number of partitions on the Kafka topic. Ideally, the number of partitions can be equal to the number of CPU cores allocated to the executors. For example, if you set `spark.executor.instances` to 6 and `spark.executor.cores` to 3, there are 18 cores allocated. Then set the number of Kafka partitions to 18, so that there are 18 parallel tasks to read from the Kafka Source.
  For example, you can use the following command to specify the number of partitions:

  ```
  ./ kafka-topics.sh --create --zookeeper
  zookeeper_host_name1:zookeeper_port_number ,zookeeper_host_name2:zookeeper_port_number,zoo
  keeper_host_name3:zookeeper_port_number --replication-factor 1 --partitions 18 --topic
  NewOSConfigSrc
  ```
- Ensure that the Kafka producer is publishing messages to every partition in a load balanced manner.
- Reduce the number of network hops between Intelligent Streaming and the Kafka cluster. Ideally the Kafka broker must be on the same machine as the data node or the Kafka cluster can run on its own machines with a zero latency network.
- Configure the `batch.size` and `linger.ms` properties to increase throughput. For each partition, the producer maintains buffers of unsent records. The `batch.size` property specifies the size of the buffer. To accumulate as many messages as possible in the buffer, configure a high value for the `batch.size` property.
  By default, the buffer sends messages immediately. To increase the time that the producer waits before sending messages in a batch, set the `linger.ms` property to 5 milliseconds.

General Recommendations

Consider the following general recommendations for tuning mapping performance:

- Run a Spark job with WARNING level of logging on every data node. To specify the log level, add a parameter to the infaspark.executor.extraJavaOptions option in the `hadoopEnv.properties` file with the following value:
  `-Dlog4j.configuration=file:/var/log/hadoop-yarn/log4j.properties`

- Do not run the NodeManager role on the Resource Manager node because the processing might become extremely slow and it might create a straggler node.
- Look for straggler nodes in the cluster. Either remove them from cluster or fix the reason for slowness.
- Verify that non-business critical data does not get ingested into Intelligent Streaming. This can be any data that is of no value to the mapping or downstream systems.
- Configure an appropriate batch interval because the batch interval affects performance. A value that is too small adversely affects performance. The recommended value is 20 seconds.
- To understand garbage collection usage, enable verbose garbage collection for both the executor and the driver.
  To enable verbose garbage collection, add the following configuration in the `hadoopEnv.properties` file:

```
infaspark.executor.extraJavaOptions=-Dlog4j.configuration=file:/var/log/hadoop-yarn/
log4j.properties
-Djava.security.egd=file:/dev/./urandom -XX:MaxMetaspaceSize=256M -verbose:gc -XX:
+PrintGCDateStamps
-XX:+PrintGCTimeStamps -XX:+PrintGCDetails -Xloggc:/var/log/hadoop-yarn/
jvm_heap_usage_executor_%p.log
-XX:+TraceClassUnloading -XX:+TraceClassLoading -XX:+HeapDumpOnOutOfMemoryError

infaspark.driver.client.mode.extraJavaOptions=-Djava.security.egd=file:/dev/./urandom -
XX:MaxMetaspaceSize=256M
-verbose:gc -XX:+PrintGCDateStamps -XX:+PrintGCTimeStamps -XX:+PrintGCDetails
-Xloggc:/var/log/hadoop-yarn/jvm_heap_usage_Driver_client_%p.log -XX:+TraceClassUnloading
-XX:+TraceClassLoading
-XX:+HeapDumpOnOutOfMemoryError
```

Monitor Your Production Environment

In the production environment, monitor the CPU, disk, and network usage.

Perform the following tasks based on the scenario:

- If the usage hits 75% constantly because of data processing needs, then add a data node to the cluster, add additional CPU cores, or additional capacity to the network interface card.
- If the Hadoop distribution that you use is Cloudera or Hortonworks, use the web UI to monitor the health of the cluster. Verify that the cluster components are healthy. To avoid outages, address all unhealthy components immediately. If required, add additional data nodes to the cluster.

Authors

Vidya Vasudevan

Shahbaz Hussain

Sreelal S L